Java SOLID principles

What SOLID is, what each of the five principles means, and how the good and bad versions of each look in code.

Contents

1. What is SOLID?
    - Single Responsibility Principle (SRP)
    - Open/Closed Principle (OCP)
    - Liskov Substitution Principle (LSP)
    - Interface Segregation Principle (ISP)
    - Dependency Inversion Principle (DIP)
2. What problems do SOLID principles solve?
3. What does "one reason to change" mean?

1. What is SOLID?

SOLID is a set of **five object-oriented design principles** that help us write code that stays maintainable, scalable, flexible, and easy to test. The term was introduced by Robert C. Martin (Uncle Bob).

Principle	Full form	Purpose
S	Single Responsibility Principle (SRP)	A class should have only one reason to change
O	Open/Closed Principle (OCP)	Software should be open for extension but closed for modification
L	Liskov Substitution Principle (LSP)	A subclass should replace its parent without changing the program's behaviour
I	Interface Segregation Principle (ISP)	Clients should not be forced to depend on methods they do not use
D	Dependency Inversion Principle (DIP)	Depend on abstractions, not concrete implementations

Single Responsibility Principle (SRP)

A class should have **only one responsibility**, one reason to change.

Bad: one class handles three unrelated jobs.

```
class UserService {
    void saveUser() { }
    void sendEmail() { }
    void generateReport() { }
}
```

Good: each job gets its own class.

```
class UserService {
    void saveUser() { }
}
class EmailService {
    void sendEmail() { }
}
class ReportService {
    void generateReport() { }
}
```

Open/Closed Principle (OCP)

A class should be **open for extension** but **closed for modification**.

Bad: every new payment type forces us to edit existing code.

```
class PaymentService {
    void pay(String type) {
        if ("UPI".equals(type)) {
        } else if ("CARD".equals(type)) {
        }
    }
}
```

Good: add a new payment type by writing a new class, not by touching the old one.

```
interface Payment {
    void pay();
}
class UPIPayment implements Payment {
    public void pay() { }
}
class CardPayment implements Payment {
    public void pay() { }
}
```

Liskov Substitution Principle (LSP)

A subclass should be usable wherever its parent is expected, without breaking the program.

Good: a `Dog` can stand in for an `Animal` with no surprises.

```
class Animal {
    void move() {
    }
}
class Dog extends Animal {
    @Override
    void move() {
        System.out.println("Running");
    }
}
```

Interface Segregation Principle (ISP)

Do not force a class to implement methods it does not need.

Bad: a robot worker does not eat, but this interface makes it implement `eat()`.

```
interface Worker {
    void work();
    void eat();
}
```

Good: split the interface so each class implements only what applies to it.

```
interface Workable {
    void work();
}
interface Eatable {
    void eat();
}
```

Dependency Inversion Principle (DIP)

High-level modules should depend on **abstractions**, not concrete implementations.

Bad: `UserService` is tightly coupled to `EmailService`.

```
class UserService {
    private EmailService emailService = new EmailService();
}
```

Good: depend on an interface, so `UserService` works with any notification implementation.

```
interface NotificationService {
    void send();
}
class EmailService implements NotificationService {
    public void send() {
    }
}
class UserService {
    private final NotificationService notificationService;
    UserService(NotificationService notificationService) {
        this.notificationService = notificationService;
    }
}
```

Summary:

Principle	One-line rule
SRP	One class, one responsibility
OCP	Extend behaviour without modifying existing code
LSP	Child classes should safely replace parent classes
ISP	Keep interfaces small and focused
DIP	Depend on interfaces, not implementations

2. What problems do SOLID principles solve?

They tackle the common design problems that make code painful over time: **tight coupling, poor maintainability, code duplication, rigid designs, and hard-to-test code**. Applied well, they give us a system that is easier to extend, understand, and maintain.

3. What does "one reason to change" mean?

It means a class should answer to **only one actor or one kind of requirement**. It is not about having one method or one job; it is about who would ever ask us to change it.

Bad: one class serving three different stakeholders.

```
class EmployeeService {
    void saveEmployee() { }
    void generateSalary() { }
    void sendEmail() { }
}
```

Ask why this class could change, and we get three separate answers:

- HR changes how employee information is stored, so `saveEmployee()` changes.
- Finance changes the salary calculation, so `generateSalary()` changes.
- Marketing changes the email template, so `sendEmail()` changes.

Three different reasons to change means it breaks SRP.

Better: one class per stakeholder.

```
class EmployeeService {
    void saveEmployee() { }
}

class PayrollService {
    void generateSalary() { }
}

class EmailService {
    void sendEmail() { }
}
```

Now an HR change touches only `EmployeeService`, a Finance change only `PayrollService`, and a Marketing change only `EmailService`. Each class has exactly one reason to change.